

Week 2: Python, Data, and ML Refresher

Seven-day learning record and classical ML checkpoint

Overview

During Week 2, I completed a seven-day refresher covering Python, data processing, machine-learning workflows, model evaluation, and error analysis. The week concluded with a complete classical text-classification checkpoint.

Day 1: Reusable Text-Cleaning Functions

Task: Build reusable functions for cleaning multilingual text.

The text-cleaning module performs:

- Unicode NFC normalization
- Missing-value handling
- Whitespace normalization
- URL replacement with `<URL>`
- Hashtag preservation as `<HASHTAG> hashtag_word`
- Repeated-punctuation limiting
- Exact duplicate removal
- UTF-8 file reading and writing

Bengali text, English text, code-mixed content, and emojis are preserved.

```
project_01_classical_baseline/  
├── data/  
│   ├── sample_raw_texts.txt  
│   └── sample_cleaned_texts.txt  
├── notebooks/  
│   └── 01_text_cleaning_test.ipynb  
└── src/  
    └── text_cleaning.py
```

Day 2: NumPy Vectors and Cosine Similarity

Notebook: `02_numpy_vectors.ipynb`

Topics covered:

- NumPy arrays, dimensions, and shapes
- Manual and NumPy dot-product calculations
- Manual vector-magnitude calculation
- Manual and NumPy cosine similarity
- Reusable similarity functions
- Text representation using bag-of-words vectors
- Pairwise similarity matrices
- Zero-vector error handling
- Practice exercises with automatic assertions

```
project_01_classical_baseline/  
├── notebooks/  
│   ├── 01_text_cleaning_test.ipynb  
│   └── 02_numpy_vectors.ipynb
```

Day 3: Pandas and Data-Quality Analysis

Notebook: 03_pandas_data_quality.ipynb

Topics covered:

- Loading CSV data with pandas
- Inspecting shape, columns, data types, and missing values
- Filtering valid records
- Grouping data by label and platform
- Detecting missing or empty text
- Detecting duplicate text
- Finding invalid labels and unexpected platforms
- Producing a compact data-quality report
- Automated verification with assertions
- Independent practice exercises

```
project_01_classical_baseline/  
└─ notebooks/  
    └─ 01_text_cleaning_test.ipynb  
    └─ 02_numpy_vectors.ipynb  
    └─ 03_pandas_data_quality.ipynb
```

Day 4: Leakage-Safe Machine-Learning Workflow

Notebook: 04_ml_workflow.ipynb

Topics covered:

- Train, validation, and test splits
- Purpose of using three separate datasets
- Stratified data splitting
- Data leakage
- TF-IDF leakage prevention
- `DummyClassifier` baseline
- TF-IDF with Logistic Regression
- Scikit-learn `Pipeline`
- Training and validation performance comparison
- Detection of possible overfitting
- Final test-set evaluation
- Importance of keeping the test set untouched
- Automatic assertions
- Independent exercises

The notebook produces a clean and leakage-safe train/validation/test machine-learning pipeline.

```
project_01_classical_baseline/  
└─ notebooks/  
    └─ 01_text_cleaning_test.ipynb  
    └─ 02_numpy_vectors.ipynb  
    └─ 03_pandas_data_quality.ipynb  
    └─ 04_ml_workflow.ipynb
```

Day 5: Classical Text-Classification Baseline

Notebook: 05_classical_baseline.ipynb

This notebook contains the first complete classical machine-learning text-classification experiment.

Final Baseline Results

```
Accuracy: 0.833  
Precision: 0.750  
Recall: 1.000  
F1: 0.857  
Macro-F1: 0.829
```

Experiment Components

Component	What it does
Dataset	Creates factual (0) and clickbait (1) text examples
Stratified split	Creates 70% training, 15% validation, and 15% test sets while maintaining class balance
TF-IDF	Converts text into numerical features
Logistic Regression	Learns to classify factual and clickbait text
(1,1)	Tests unigrams
(1,2)	Tests unigrams and bigrams
(1,3)	Tests unigrams, bigrams, and trigrams
Validation	Compares the three TF-IDF configurations
Model selection	Selects the configuration with the best validation Macro-F1
Test evaluation	Evaluates the selected model on unseen data
Accuracy	Measures the overall percentage of correct predictions
Precision	Measures how often a clickbait prediction is correct
Recall	Measures how many actual clickbait examples are identified
F1	Balances precision and recall
Macro-F1	Gives equal importance to both classes
JSON saving	Saves the final results to <code>day_05_baseline_metrics.json</code>

```

project_01_classical_baseline/
├── notebooks/
│   ├── 01_text_cleaning_test.ipynb
│   ├── 02_numpy_vectors.ipynb
│   ├── 03_pandas_data_quality.ipynb
│   ├── 04_ml_workflow.ipynb
│   └── 05_classical_baseline.ipynb
├── results/
│   └── day_05_baseline_metrics.json
└── src/
    └── text_cleaning.py

```

Day 6: Model Evaluation and Error Analysis

Notebook: `06_evaluation_error_analysis.ipynb`

Key Findings

Accuracy: 0.833
Macro-F1: 0.829

True negatives: 2
False positives: 1
False negatives: 0
True positives: 3

The model made only one incorrect prediction:

Researchers publish results of health study

Topics covered:

- Accuracy and Macro-F1
- Confusion matrix
- True positives and true negatives
- False positives and false negatives
- Precision, recall, F1, and support for each class
- Test-prediction inspection
- Misclassified-example analysis
- TF-IDF vocabulary limitations
- Prediction-probability interpretation
- Automatic generation of a one-page error-analysis report

```
project_01_classical_baseline/
├── notebooks/
│   ├── 05_classical_baseline.ipynb
│   └── 06_evaluation_error_analysis.ipynb
├── results/
│   ├── day_05_baseline_metrics.json
│   └── day_06_error_analysis.md
```

Day 7: Project 1 Checkpoint

Notebook: 07_project_01_checkpoint.ipynb

The final checkpoint combines the complete workflow:

- Loading and checking the dataset
- Creating stratified data splits
- Comparing TF-IDF configurations
- Selecting the best model
- Evaluating the model on unseen test data
- Saving the trained model
- Exporting predictions, metrics, and the confusion matrix

Final Checkpoint Results

Selected model: Word TF-IDF (1,1) + Logistic Regression
Validation Macro-F1: 0.829
Test accuracy: 0.833
Test Macro-F1: 0.829

Final Project Structure

```
project_01_classical_baseline/
├── data/
│   └── practice_clickbait.csv
├── notebooks/
│   ├── 05_classical_baseline.ipynb
│   ├── 06_evaluation_error_analysis.ipynb
│   └── 07_project_01_checkpoint.ipynb
├── models/
│   └── project_01_classical_baseline.joblib
├── results/
│   ├── project_01_checkpoint_metrics.json
│   ├── project_01_test_predictions.csv
│   └── project_01_confusion_matrix.png
```

Week 2 Completion Status

Item	Status
Project 1 checkpoint	Passed
Week 2	Complete

Repository: [View the GenAI Project on GitHub](#)